

StoryFlowDSL - A Model-Driven Engineering Pipeline for Interactive Branching Narratives

Model-Driven Engineering 2025/2026

Cláudia Faria nº73777

Tomás Ferreira nº73079

Índice

STORYFLOWDSL - A MODEL-DRIVEN ENGINEERING PIPELINE FOR INTERACTIVE BRANCHING

NARRATIVES.....	1
1. INTRODUCTION & DOMAIN ANALYSIS	3
1.1 Context and Motivation.....	3
1.2 Stakeholder Analysis.....	4
1.3 Use Case Scenarios.....	4
UC1 — Conditional Scene Unlock (Investigation Gate)	4
UC2 — Butterfly Effect/Cross-Episode Variable Persistence	5
UC3 — Inventory-Gated Transition (Key and Lock)	5
UC4 — Probabilistic / Dice-Rule Transition	6
UC5 — Branching Endings (Sacrifice vs. Save)	6
1.5 Non-Functional Requirements	8
2. METAMODELLING	9
2.1 High-Level Language (HLL) Metamodel.....	9
2.2 Pivot Language Metamodel.....	10
2.3 Justification of Design Choices.....	11
3. GRAPHICAL EDITOR (SIRIUS).....	12
4. EVL VALIDATION.....	14
4.1 Validation Strategy	14
4.2 Constraint Definitions.....	14
4.3 Validation Results	15
5. MODEL TRANSFORMATIONS (ETL).....	16
5.1 Traceability.....	16
6. CODE GENERATION (EGL).....	17
7. FORMAL VERIFICATION.....	19
7.1 Translation Strategy: EGL Template for NuSMV.....	19
7.2 Properties Verified.....	19
7.3 Verification Results	20
8. TEST SUITE	21
8.1 TEST SUITE OVERVIEW	21
8.2 T1 - Positive Test Cases.....	22
8.3 T2 - Metamodel Failure Test Cases	23
8.4 T3 - EVL Failure Test Cases.....	23
8.5 T4 - Semantic Failure (Formal Verification) Test Cases.....	24
9. REFLECTION ON AI USAGE	25
9.1 Phases Where AI Was Most Useful.....	25
9.2 Phases Where AI Was Less Useful or Produced Incorrect Output	25
9.3 How the Prompting Strategy Evolved.....	26
9.4 The Role of Generative AI in Language Engineering.....	26
10. CONCLUSION.....	27

1. Introduction & Domain Analysis

1.1 Context and Motivation

Domain-Specific Languages are among the most expressive tools in the Software Language Engineering (SLE) toolkit. When carefully designed, they allow domain experts to express solutions in the vocabulary of their own field, and allow engineers to generate complex, formally analysed artifacts from high-level models without requiring domain experts to understand the underlying implementation.

StoryFlowDSL is a DSL designed for the domain of interactive branching narratives in which a reader or player makes choices that alter the trajectory of the narrative, the state of the world, and the set of reachable endings. The domain is characterised by a directed graph of narrative scenes connected by guarded transitions, whose enabling conditions depend on the accumulated state of player-controlled variables and inventory items.

The reference system informing this project is *Life is Strange*, a five-episode narrative adventure in which the protagonist's choices accumulate across episodes through a persistent state of boolean flags and inventory items. This butterfly-effect design pattern, where in an action in Episode 1 may silently alter the options available in Episode 4, provides a richly documented and mechanically precise reference for modelling long-range conditional dependencies between narrative scenes. It is ideally suited to a formal MDE approach: the high-level language captures creative authoring intent, the pivot language exposes the underlying state machine, and formal verification provides correctness guarantees that play testing alone cannot.

The pipeline developed in this project traverses the complete DSL development lifecycle: domain analysis, metamodeling, graphical editing, static validation, model-to-model transformation, model-to-text generation, and formal verification. Each phase produces concrete, executable artifacts. The final output of the pipeline is a self-contained, browser-playable HTML interactive story runner generated entirely from a high-level author model, with zero manual coding by the narrative author.

1.2 Stakeholder Analysis

Stakeholder	Role and Interest
Game Designer	Primary user of the high-level DSL. Defines scenes, choices, conditions, and endings without programming. Needs expressive, domain-vocabulary notation.
Programmer	Consumes the pivot language or generated artifacts to implement or embed the story in a game engine.
QA / Playtester	Executes generated stories to verify reachability of all endings and absence of dead-end states.
Accessibility Reviewer	Checks that all narrative paths are explorable and that no ending is accidentally locked.
Tool Developer (MDE Engineer)	Designs and maintains the DSL, metamodel, transformations, and verification scripts.

1.3 Use Case Scenarios

UC1 — Conditional Scene Unlock (Investigation Gate)

Reference in Life is Strange

Max can only confront Juliet about the fake text message if she has already found the note in Dana's room (variable `found_note` = true). Without it, the confrontation choice is hidden from the dialogue menu.

Situation:

Max is in the dormitory corridor. The scene "Confront Juliet" has a single incoming transition guarded by `found_note == true`.

Configuration:

- Scene **corridor_dorm** has two outgoing choices: "Ignore the situation" (unconditional) and "Confront Juliet" (condition: `found_note == true`).
- Variable **found_note** is set to true in scene **dana_room** when the player examines the letter item.

Expected Behaviour:

- If `found_note == false`: the choice "Confront Juliet" is invisible or disabled.
- If `found_note == true`: both choices are available: "Ignore the situation" and "Confront Juliet".

UC2 — Butterfly Effect/Cross-Episode Variable Persistence

Reference in Life is Strange

Watering Lisa's plant in Episode 1 sets `plant_watered = true`. In Episode 2, the engine checks this flag to determine the plant's visual state.

Situation:

The story spans two chapters. An action in Chapter 1 sets a flag and in Chapter 2 reads that flag.

Configuration:

- Scene **dorm_room_ep1** contains an optional action "Water the plant". If the player chooses to water the plant then `plant_watered = true`.
- Scene **hallway_ep2** has two outgoing transitions to `alive_plant_scene` and `dead_plant_scene`, guarded by `plant_watered == true` and `plant_watered == false` respectively.

Expected Behaviour:

- If the player watered the plant, **hallway_ep2** routes to `alive_plant_scene`.
- Otherwise, `dead_plant_scene` is reached.

UC3 — Inventory-Gated Transition (Key and Lock)

Reference in Life is Strange

To enter the Dark Room in Episode 4, Max must first obtain Frank's RV key by either winning the negotiation peacefully or confronting him. The door transition is gated on `has_rv_key` in inventory.

Situation:

A locked door separates two areas. Max must possess the correct key item to be able to do the transition.

Configuration:

- Transition `enter_dark_room` has a condition: inventory contains `rv_key`.
- Item `rv_key` is added to inventory by scene **frank_negotiation** when the player selects the peaceful resolution.

Expected Behaviour:

- Without `rv_key`, the transition is blocked and the player must return to obtain the item.
- With `rv_key`, the transition fires and Max enters the dark room.

UC4 — Probabilistic / Dice-Rule Transition

Reference in Life is Strange

Some skill-check or tension scenes can be modelled as probabilistic outcomes.

Situation:

Max attempts a risky action. The outcome is determined by a dice roll with defined success probability.

Configuration:

- Transition `risky_jump` has a DiceRule: min 1, max 6 with a success threshold of 4 (success on 4-6).
- On success: scene `safe_landing`. On failure: scene `caught_by_guard`.

Expected Behaviour:

- The engine rolls a value in [1, 6].
- Value ≥ 4 leads to `safe_landing`. Value < 4 leads to `caught_by_guard`.

UC5 — Branching Endings (Sacrifice vs. Save)

Reference in Life is Strange

The final choice of the game, sacrifice Chloe to save Arcadia Bay, or sacrifice the town to save Chloe. This is a binary fork leading to two mutually exclusive ending scenes.

Situation:

The story is reaching an end and there's two endings available.

Configuration:

- Scene `lighthouse_decision` has two unconditional outgoing choices: "Sacrifice Chloe" -> `ending_sacrifice_chloe` and "Save Chloe" -> `ending_save_chloe`.
- Both ending scenes are marked `isEnding = true` with no outgoing transitions.

Expected Behaviour:

- Each choice leads to exactly one terminal scene.
- The formal verifier confirms both endings are reachable from the start scene and that no further transitions exist beyond them.

1.4 Functional Requirements

ID	Requirement
FR-01	The language must allow the author to declare a set of named scenes, exactly one of which is marked as the start scene. (UC1, UC2, UC5)
FR-02	The language must allow the author to define choices on scenes, with a human-readable label and a target scene reference. (UC1, UC5)
FR-03	The language must allow the author to attach a boolean condition to a choice, referencing declared story variables and inventory items. (UC1, UC2, UC3)
FR-04	The language must allow the author to declare story-scoped variables with a name, type (Boolean, Integer, String), and default value. (UC1, UC2)
FR-05	The language must allow the author to declare inventory items at story scope and express 'has item' conditions on choices. (UC3)
FR-06	The language must allow the author to attach state effects to choices: variable assignment (SET_VARIABLE) and item acquisition (ADD_ITEM). (UC2, UC3)
FR-07	The language must allow the author to attach a probabilistic dice rule to a choice, specifying a numeric range and a success threshold, with separate success and failure target scenes. (UC4)
FR-08	The language must allow the author to declare one or more terminal ending scenes, marked so that no outgoing choices are permitted on them, and guarantee that at least one ending scene is reachable from the start scene. (UC5)
FR-09	The language must allow the author to group scenes into ordered chapters or acts for structural organisation across long narratives. (UC2, UC5)

1.5 Non-Functional Requirements

ID	Requirement
NFR-01 (Usability)	The DSL syntax must be readable by non-programmer narrative authors: concept names must match domain vocabulary (Scene, Choice, Item, not Node, Edge, Token). A new user should be able to model a 5-scene story in under 30 minutes without reading more than the README.
NFR-02 (Validation)	Models that violate structural constraints (duplicate scene ids, missing start scene, invalid item references) must be rejected before any transformation or code generation is attempted, with clear, error messages.
NFR-03 (Formal Correctness)	The formal verification step must be capable of detecting at least: scenes from which no ending is reachable (dead-ends), endings that are permanently unreachable from the start, and any cycle from which no ending scene is reachable,
NFR-04 (Maintainability)	Adding a new type of transition effect (e.g. a new variable type or a new condition operator) must require changes only to the metamodel and EVL/ETL rules.
NFR-05 (Traceability)	Every generated artifact element (scene function in JS, state in NuSMV) must be traceable to its source scene/choice in the high-level model, either via naming convention or explicit trace links in the ETL transformation.
NFR-06 (Portability of Generated Artifacts)	The HTML/JS generated story runner must be self-contained in a single file and must execute without a local server in any modern browser, requiring no build step from the end user.

2. Metamodelling

2.1 High-Level Language (HLL) Metamodel

The HLL metamodel was designed according to two guiding principles: it must be expressive enough to cover all five use cases, and it must use the vocabulary of the narrative domain so that non-programmer authors can construct models without consulting technical documentation.

2.1.1 Root Structure: Story → Chapter → Scene

The HLL root element is Story, which aggregates one or more Chapter elements, each of which contains one or more Scene elements. This three-level containment hierarchy reflects the authoring vocabulary of *Life is Strange* (five episodes, each with multiple scenes) and allows authors to organise large narratives without losing semantic clarity. Chapter elements carry a strictly positive, unique order integer to enforce a canonical narrative sequence.

Variables (Variable), inventory items (inventoryitem), and characters (Character) are declared at Story scope rather than at scene scope. This design decision mirrors the cross-episode persistence of *Life is Strange*: `rv_key` is a global narrative state that must be readable by any scene in any chapter, not just the scene that acquires it.

2.1.2 Scene

Scenes use boolean flags **isStart** and **isEnding** rather than specialised subclasses. This keeps the metamodel flat and makes instance models simpler to write and to read. The EVL constraints (exactly one start, no outgoing choices on endings) impose the required semantics without requiring the extra classes.

Ending scenes additionally carry an optional **endingType:EndingType** enumeration (GOOD/BAD/NEUTRAL), which the authors can classify the narrative outcomes.

2.1.3 Choice, Condition, Effect, and dicerule

The **Choice** class bundles the label, the target scene reference, an optional **Condition**, an optional **DiceRule**, and zero or more **Effects**.

The Condition class stores a guard expression as a plain estring expression field (e.g., "inventory contains rv_key" or "found_note == true"), alongside cross-references to the specific Variable and inventoryitem instances the expression mentions (referencedvars, referenceditems). This dual representation is intentional: the expression field is what the author reads and edits, while the cross-reference fields allow EVL to validate that all referenced entities are declared at story scope.

The Effect class represents a state mutation applied when a choice is taken. It carries a typed effecttype enumeration (SET_VARIABLE, ADD_ITEM, REMOVE_ITEM), a value estring and optional cross-references targetvar and targetitem that EVL uses to verify consistency.

The dicerule class models probabilistic branching. It carries:

- Min : eint and max : eint - the bounds of the random draw.
- Successthreshold : eint - the minimum roll value for the success branch.
- Onsuccess : Scene and onfailure : Scene - the two target scenes.

This design makes the dicerule's choice explicit at the HLL level, which is both narratively legible and structurally important for the ETL transformation.

2.2 Pivot Language Metamodel

The Pivot language metamodel is designed for the machine, not the author. Its purpose is to present the story in a form to formal verification and code generation, stripping away all narrative metadata and exposing a clean, flat state-transition system.

2.2.1 narrativegraph: Flattening to Node + Edge

The Pivot root is narrativegraph, which contains a flat collection of Node and Edge elements. The three-level hierarchy (Story → Chapter → Scene) is completely dissolved. Chapter information is deliberately discarded in the Pivot because it carries no operational semantics: the chapter sequence is an authoring scaffold, not an execution constraint. Retaining it would add noise to the formal model and complicate nusmv translation without adding expressive power.

2.2.2 Node

Each HLL Scene becomes a Node in the Pivot. The Node retains only the operationally relevant attributes id, label, isStart, isAccepting (the pivot term for isEnding), and endingType. Description text, character associations, and other authoring metadata are dropped.

2.2.3 Edge: Unifying Choice, DiceRule, and Effects

Each HLL Choice becomes an Edge in the pivot. An Edge aggregates:

- A Guard (renamed from Condition) with expression and variable/item cross-references.
- Zero or more Actions (renamed from Effects) typed by ActionType (SET_VAR, ADD_ITEM, REMOVE_ITEM).
- An optional PivotDiceRule carrying successTarget and failureTarget Node references.

The HLL's DiceRule.onSuccess/onFailure scene references are flattened into the Edge's PivotDiceRule because in the pivot graph, all successors of a node are represented as outgoing edges, whether deterministic or probabilistic.

2.3 Justification of Design Choices

2.3.1 Boolean Flags vs. Subclassing for Start and Ending Scenes

A natural object-oriented instinct would be to define startscene and endscene as subclasses of Scene, with the Ecore type system enforcing the relevant constraints structurally. This approach was considered and rejected for the following reasons:

- **Metamodel complexity:** Introducing subclasses would require ETL rules to handle multiple Scene subtypes, increasing transformation complexity disproportionately. With boolean flags, the ETL Scene rule is uniform: every scene becomes a Node; the isstart and isaccepting flags are simply copied. EVL constraints — exactlyonestartscene, endscenehasnochoices — handle the semantic invariants cleanly and with legible error messages.
- **Instance model readability:** Authors constructing XML models (or using the Sirius editor) encounter a single Scene concept. Boolean flags are a familiar notational shorthand in domain modelling and require no understanding of type hierarchies.
- **Flexibility for future extension:** If a future requirement introduces, for example, a "mid-point save scene" concept, it can be modelled as an additional flag or enumeration value rather than requiring a new subclass and the attendant modifications to all ETL rules.
- **Precedent in the EVL constraint set:** The existing nine EVL constraints already enforce all the required semantic invariants on isstart and isending. Ecore subtyping would duplicate these constraints in a less legible form.

2.3.2 The HLL/Pivot Boundary

The boundary between the HLL and Pivot was drawn according to a single criterion: *does this concept carry operational execution semantics?* If a concept is needed to evaluate

transitions, update state, or generate the nusmv model, it belongs in the Pivot. If it exists solely to support authoring clarity, it remains in the HLL and is discarded by ETL.

This criterion explains the following specific decisions:

- **Chapters are discarded:** A chapter's order integer supports the author's structural organisation but plays no role in determining which transitions are fireable.
- **Description text and characters are discarded:** These are rendered by EGL from the Pivot for the HTML runner, but as static strings — they do not affect transition logic. They are retained in the Pivot Node's label but not in any form that affects nusmv verification.
- **Dicerule is retained and promoted:** The dicerule is operationally significant: it governs which scene the player reaches. It is therefore present in both languages, but restructured in the Pivot to make both branches explicit as separate `success` and `failure` node references.
- **Condition is renamed Guard in the Pivot:** This rename reflects the shift from an authoring concept (a condition an author writes) to a formal concept (a guard that nusmv evaluates). The renaming is deliberately chosen to signal the semantic shift, even though the underlying structure is similar.

3. Graphical Editor (Sirius)

A graphical editor for the HLL was developed using the Sirius Workbench. The `.odesign` specification defines a sophisticated visual language tailored to the domain:

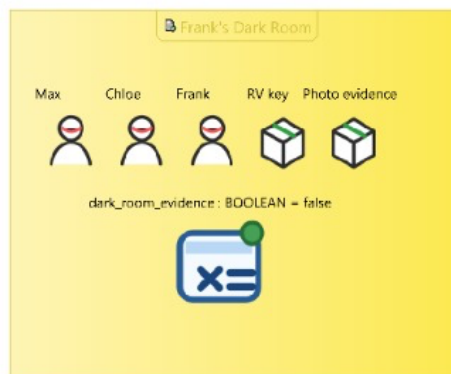
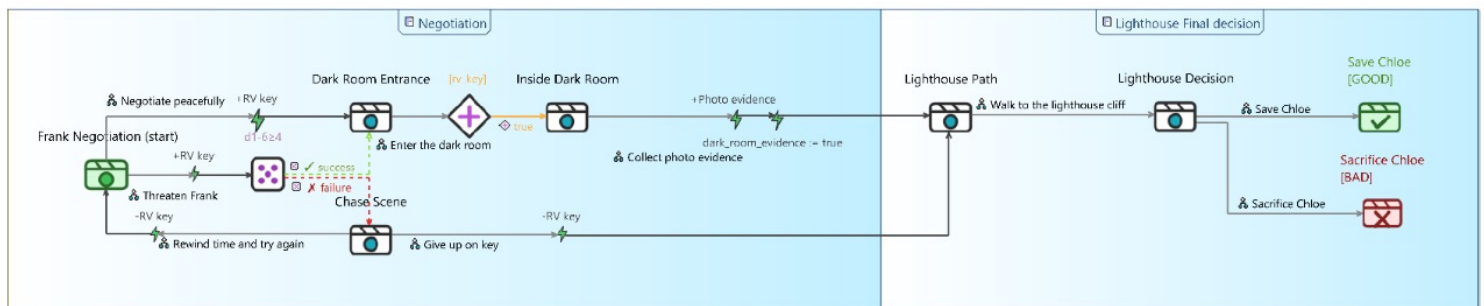
- **Scenes and Chapters:** Chapters are rendered as container nodes (`ChapterContainer`), visually grouping their constituent scenes. Scenes are represented using custom SVG icons (`WorkspaceImageDescription`). The graphical representation changes dynamically based on the scene's state: start scenes receive a specific marker, while ending scenes are highlighted with distinct borders and badges (`GOOD` / `BAD` / `NEUTRAL`) depending on their `endingType`.
- **Resources Container:** Global narrative state - variables, characters, and inventory items - are neatly segregated into a dedicated `ResourcesContainer` (rendered with a distinct yellow styling), keeping them visually distinct from the narrative flow.
- **Complex Choice Chains:** The representation of choices is highly dynamic. A simple, unconditional choice is rendered as a standard directed edge between two scenes. However, if a choice contains conditions, effects, or dice rules, it is expanded into a chain of intermediate nodes (`ConditionNode`, `EffectNode`, `DiceRuleNode`) connected by

sequence edges. This allows authors to visually trace the exact execution flow of a complex transition.

- Dice Rules: Probabilistic branches (DiceRuleNode) split the flow into two explicitly rendered edges (DiceSuccessEdge and DiceFailureEdge), both styled with dashed lines to clearly distinguish probabilistic routing from deterministic transitions.

Rather than cluttering the element palette with every concept, the palette only exposes the core structural elements (Chapter, Scene, Choice, Variable, Item, Character). Advanced mechanics (Conditions, Effects, and Dice Rules) are injected via a contextual popup menu on the Choice edges. This menu triggers custom Java extensions (Services.java) that display dialog prompts, validate input formats (e.g., verifying $\min < \max$ for dice rules), and safely instantiate the underlying Ecore elements.

This editor satisfies NFR-01: a narrative author with no Eclipse experience can model a complex branching story exclusively through graphical drag-and-drop authoring and contextual menus, without ever editing XML directly.



4. EVL Validation

4.1 Validation Strategy

EVL (Epsilon Validation Language) validation serves as a mandatory gate between the HLL instance model and the ETL transformation. No transformation is attempted until all EVL constraints pass. This design decision reflects **NFR-02** and prevents malformed or semantically inconsistent models from propagating errors into downstream pipeline phases, where they would be significantly harder to diagnose.

The validation strategy distinguishes two categories of constraint:

- **Structural invariants:** constraints on the shape of the model that Ecore's multiplicity system cannot express. For example, Ecore can enforce that a Story contains one or more Scene elements, but it cannot enforce that exactly one of those scenes has `isstart = true`.
- **Semantic domain rules:** constraints that encode narrative correctness, such as the requirement that an ending scene has no outgoing choices, or that a dicerule has a valid numeric range.

Nine constraints were defined across five EVL contexts: Story, Chapter, Scene, dicerule, and Effect.

4.2 Constraint Definitions

ID	Name	Context	Description
C1	Exactlyonestartscene	Story	Exactly one scene across all chapters must have <code>isstart = true</code> .
C2	Uniquechapterorder	Story	No two chapters may share the same order value.
C3	Chapternotempty	Chapter	Every chapter must contain at least one scene.
C4	Endingscenehasnochoices	Scene	A scene with <code>isending = true</code> must have zero outgoing choices.
C5	Nonendingscenehaschoices	Scene	A scene with <code>isending = false</code> must have at least one outgoing choice.

ID	Name	Context	Description
C6	Uniquesceneid	Story	No two scenes across all chapters may share the same id value.
C7	Startcannotbeending	Scene	A scene may not have both isstart = true and isending = true simultaneously.
C8	Validdicerange	Dicerule	Min < max and min ≤ successthreshold ≤ max must all hold.
c9	Effecttargetconsistency	Effect	A SET_VARIABLE effect must reference a targetvar; an ADD_ITEM or REMOVE_ITEM effect must reference a targetitem.

4.3 Validation Results

All nine constraints were executed against three instance models:

- **Storydanamodel_hll.xmi** (Dormitory Investigation): 0 violations. All structural and semantic invariants hold.
- **Storydarkroommodel_hll.xmi** (Frank's Dark Room & Lighthouse): 0 violations. The validdicerange constraint (C8) correctly passes for a dicerule with min=1, max=6, successthreshold=4.
- **Storyinvalidmodel_hll.xmi** (Broken Story - T3 test case): 7 violations detected, covering C1 (two start scenes), C2 (duplicate chapter order), C4 (ending scene with outgoing choice), C5 (non-ending scene with no choices), C6 (duplicate scene id), C8 (invalid dicerule range), and C9 (inconsistent effect target). Every deliberately introduced violation was correctly flagged.

5. Model Transformations (ETL)

The ETL (Epsilon Transformation Language) transformation converts a validated HLL instance model into a Pivot narrativegraph instance model. This transformation is the structural centrepiece of the pipeline: its completeness determines the correctness of all downstream artifacts.

The transformation is organised around three primary ETL rules:

- **Rule scenetonode:** Transforms each HLL Scene into a Pivot Node. The rule copies `isstart` directly and maps `isending` to `isaccepting`. The scene's name attribute becomes the Node's id — the primary traceability anchor. All chapter-level containment is dissolved: the output is a flat collection of nodes in the narrativegraph.
- **Rule choicetoedge:** Transforms each HLL Choice into a Pivot Edge. The rule uses `.equivalent()` to resolve the Choice.target scene reference to its corresponding Pivot Node, preserving structural links across the transformation boundary. If the choice carries a Condition, a corresponding Guard is created on the edge, retaining the expression string and resolving variable and item cross-references via `.equivalent()`. If the choice carries Effects, corresponding Action elements are created on the edge with their `actiontype` mapped from the HLL `effecttype` enumeration.
- **Rule diceruletoedge:** This rule handles the structural asymmetry between the HLL and Pivot dicerule representations. In the HLL, a dicerule is a sub-element of a Choice with two separate scene references (`onsuccess`, `onfailure`). In the Pivot, both branches must be represented as explicit `successtarget` and `failuretarget` node references on a `pivotdicerule` element attached to the Edge. The rule resolves both target scenes via `.equivalent()` and assigns the resulting nodes accordingly.

The ETL file additionally defines a top-level narrativegraph rule that collects all transformed Node and Edge elements into the Pivot root. The `.equivalent()` operation is used consistently throughout to guarantee referential integrity across transformation rules.

5.1 Traceability

Traceability between HLL and Pivot models is maintained through two complementary mechanisms:

1. **Epsilon's native trace mechanism:** The `.equivalent()` operation records a trace link from each source element to its target equivalent. These links are exploitable programmatically in downstream ETL or EOL scripts.
2. **Explicit semantic identifiers:** Each transformed element's id attribute encodes its HLL origin in a human-readable form:

- A Pivot Node receives `id = scene.name`, making the correspondence immediately legible in the generated XML.
- A Pivot Edge receives `id = parents.scene.name + "_choice_" + choice.id`. For example, the edge `dana_room_choice_1` unambiguously documents its origin as choice 1 in scene `dana_room` of the HLL model.

These two mechanisms together ensure that any element in the Pivot XML is traceable to its exact HLL origin, both programmatically via Epsilon's trace table and visually by inspection of the generated identifier.

6. Code Generation (EGL)

The EGL (Epsilon Generation Language) template traverses the Pivot narrativegraph and generates a single, self-contained HTML interactive story runner. The generated file requires no build toolchain and executes in any modern browser without a local server, satisfying **NFR-06**.

6.1 Template Structure

The EGL template is structured around two concerns: static UI scaffolding and dynamic game engine generation.

- **Static scaffolding** produces the HTML structure (title, scene panel, choices panel, inventory display, flags display, restart button) and CSS styling. These elements are fixed for all stories and do not depend on the Pivot model.
- **Dynamic game engine** is generated by iterating over the Pivot narrativegraph. The template produces a javascript object that encodes the complete narrative graph as a JSON-like structure inlined in the generated file. Each Pivot Node becomes an entry, carrying its `id`, `label`, `description`, `isstart`, `isaccepting`, and `endingtype`. Each Pivot Edge becomes an entry in the node's `choices` array, carrying its `label`, `target node reference`, `guard expression` (if present), `actions array`, and `dicerule configuration` (if present).

6.2 Runtime Game Engine

The runtime javascript engine, also generated by EGL, implements five execution behaviours:

- **Initial state initialisation:** On page load, the engine reads the `isstart` node, sets all variables to their declared default values, and initialises the inventory as an empty set.

- **Guard evaluation:** When rendering a scene's choices, the engine evaluates each edge's guard expression against the current variable and inventory state. Choices with unsatisfied guards are rendered as disabled (greyed-out) rather than hidden, making the narrative structure visible to the player.
- **Dice rule execution:** When a choice with a dicerule is selected, the engine generates a random integer in [min, max]. If the result is $\geq$ successthreshold, the successtarget node is entered and the success-branch actions are applied, otherwise the failuretarget node is entered.
- **State persistence:** All variable assignments (SET_VARIABLE actions) and inventory mutations (ADD_ITEM, REMOVE_ITEM actions) are applied to the mutable game state object, which persists for the lifetime of the browser session. Because the generated file is self-contained, no server-side storage is required.
- **Terminal node handling:** When a node with isaccepting = true is reached, the engine suppresses the choices panel and displays a terminal badge (A Good Ending, A Bad Ending, or A Neutral Ending) based on the node's endingtype.

6.3 Execution Demonstration

The generated HTML/JS runner was validated through a complete execution of the pivot, confirming that all state transitions and UI generation requirements were met successfully. As documented in the Demonstration.pdf file provided in the 06_egl/ submission folder, the generated artifact dynamically reflects the underlying formal state:

- **Initial Stae and Guard Evaluation:** On initialization, the engine correctly loads default boolean variables (e.g., dark_room_evidence = false) and an empty inventory. The UI explicitly renders choices with unmet guard conditions (such as the choice to "Enter the Dark Room" before acquiring the key) as disabled options. This proves that abstract guard expressions are correctly translated into client-side JavaScript logic.
- **Dice Rule execution & State Persistence:** The probabilistic mechanics function flawlessly. Upon successfully rolling the DiceRule threshold when threatening Frank, the engine dynamically applies the success-branch effects. The generated UI immediately updates to reflect the new state: the RV Key is appended to the Inventory panel, and the dark_room_evidence flag is visibly toggled to true. This state persistence automatically unlocks the previously disabled choice, allowing the narrative graph traversal to proceed.
- **Terminal Node Handling:** When the player reaches an accepting node (e.g., ending_sacrifice_chloe), the engine accurately halts the traversal. The choice panel is completely suppressed, and the UI displays the correct terminal badge ([ENDING

SACRIFICE CHLOE] - A Bad Ending), validating the structural integrity of the generated pivot graph.

7. Formal Verification

7.1 Translation Strategy: EGL Template for NuSMV

Formal verification is implemented through an EGL model-to-text transformation that generates a NuSMV .smv file from the Pivot NarrativeGraph. The transformation is launched by generate_nusmv.egx, where each Pivot NarrativeGraph is transformed using the generate_nusmv.egl template. The generated file name is derived from the Pivot model resource name.

7.2 Properties Verified

- **P1 (Ending Reachability):** Every declared ending scene is reachable from the start node via at least one execution path. CTLSPEC EF node = n_<endingId>
- **P2 (Deadlock Freedom):** No reachable non-ending scene may be a deadlock or softlock. CTLSPEC AG (!is_accepting -> can_move)
- **P3 (Inventory Gate Correctness):** An inventory-gated scene cannot be entered without the required item being present. INVARSPEC !(node = n_<sceneId> & !has_<itemId>)
- **P4 (No unintended infinite loops):** From every reachable state, some accepting ending must still be reachable. This prevents unintended infinite loops and irreversible softlocks. CTLSPEC AG EF is_accepting
- **P5 (Good endings):** At least one GOOD ending must be reachable. CTLSPEC EF is_good_ending

7.3 Verification Results

All seven instance models in the test suite that passed validation were transformed to NuSMV with the generate_nusmv.egl template and checked against the five generated properties P1–P5 defined above. The table below records the verdict of every property for every model. The four positive (T1) models satisfy all five properties; each negative (T4) model deliberately violates one or more, producing the counterexamples analysed in 7.4.

Model	P1	P2	P3	P4	P5
The Lighthouse Vision (T1)	hold	hold	hold	hold	hold
The Blackwell Break-In (T1)	hold	hold	hold	hold	hold
The Train Tracks (T1)	hold	hold	n/a	hold	hold
The End of the World Party (T1)	hold	hold	hold	hold	hold
T4-01 Unreachable good ending	FAIL	hold	hold	hold	FAIL
T4-02 Softlock / deadlock	FAIL	FAIL	hold	FAIL	FAIL

Legend: **hold** - the property is verified (NuSMV reports the specification true). **FAIL** - the property is false and NuSMV returns a counterexample. **n/a** - the model has no scene that is exclusively item-locked, so the generator emits CTLSPEC AG TRUE for P3.

7.4 Counterexample Analysis

The two negative (T4) cases were built to be structurally valid and to pass all nine EVL constraints, yet to violate a behavioural property that only the model checker can detect. Each fails a different property, which shows the battery is non-redundant and that verification reaches defects beyond the scope of static validation.

T4-01 - Unreachable good ending.

The GOOD ending ending_save_everyone is reachable only through a choice guarded by has_bunker_key, and bunker_key is never produced by any action. NuSMV reports P1 (EF node = n_ending_save_everyone) and P5 (EF is_good_ending) false, while P2 and P4 hold because the BAD ending stays reachable, so there is no dead-end. The story is structurally

impeccable and EVL-clean, one start, unique identifiers, terminal endings, yet it is unwinnable. No EVL constraint expresses reachability, so this fault is invisible to static validation and is exposed only by P1/P5.

T4-02 - Softlock / deadlock (the primary counterexample).

The player walks from the corridor into dark_room, but its only exit needs exit_code, an item the story never grants. So once inside dark_room, no choice is ever available again. The NuSMV trace shows this at State 2.2: dark_room is a non-ending scene with can_move = FALSE, a permanent dead-end. P1, P4 and P5 all fail because of this same trap, and P3 stays true only because the guarded ending is never reached. EVL misses the problem entirely: it sees that dark_room has an outgoing choice and is satisfied. What EVL can't see is that the choice's guard can never be met, because that depends on inventory built up across the whole story.

8. Test Suite

8.1 Test Suite Overview

The test suite is organised into four categories covering the complete pipeline, from Ecore structural validity through formal behavioural properties. A minimum of ten test cases are required across the four categories.

Category	What It Tests	Success Criterion	Minimum
T1 — Positive	Valid models traversing the full pipeline	Conform to metamodel → pass EVL → ETL → nusmv passes → EGL produces runnable artifact	≥ 3
T2 — Metamodel Failure	Instances violating Ecore structure	Rejected by EMF validator before reaching EVL	≥ 2
T3 — EVL Failure	Structurally conformant models violating semantic constraints	Pass EMF Rejected by EVL	≥ 3
T4 — Semantic Failure (Formal Verification)	Models passing EVL but exhibiting unsafe runtime behaviour	Pass Ecore + EVL Nusmv or TINA detects violation and produces counterexample	≥ 2

8.2 T1 - Positive Test Cases

We created four positive models, each displaying a distinct combination of language features. Everyone traverses the complete pipeline: it conforms to the Ecore metamodel, passes all nine EVL constraints, is transformed by the ETL into a well-formed pivot NarrativeGraph, is generated into a playable HTML story runner by the EGL, and is verified in NuSMV with every property reported true.

T1-01: StoryLighthouseVision_hll.xmi - The Lighthouse Vision.

This story has an item gate (lighthouse_key), a dice climb, a SET_VARIABLE effect (vision_understood), an escapable loop, and two ending types (GOOD and NEUTRAL).

Result: All stages pass. NuSMV reports every specification is true: both endings reachable, no deadlock, no softlock, and the locked-scene invariant $G(\text{node} = n_lamp_room \rightarrow \text{has_lighthouse_key})$ holds.

T1-02: StoryBlackwellBreakIn_hll.xmi - The Blackwell Break-In.

This story has a combined gate (the knows_passcode variable AND the keycard item), two SET_VARIABLE effects, an ADD_ITEM effect, an escapable records-room loop, and GOOD and BAD endings.

Result: All stages pass. NuSMV confirms both endings reachable, no deadlock or softlock, and the invariant $G(\text{node} = n_sealed_cabinet \rightarrow \text{has_keycard})$ holds (the vault is never opened without the keycard).

T1-03: StoryTrainTracks_hll.xmi - The Train Tracks.

This story has an INTEGER variable (rewind_energy), an item gate (crowbar) and a variable gate (lever_freed), a dice roll whose FAILURE reaches a BAD ending, and three ending types.

Result: All stages pass. NuSMV confirms all three endings reachable (including too_late via dice failure) and no deadlock or softlock. No scene is exclusively item-locked, so P3 is emitted as CTLSPEC AG TRUE.

T1-04: StoryVortexParty_hll.xmi - The End of the World Party.

This story has a chained gate (flashlight grants hidden_drive, which gates a scene that sets truth_known, which in turn gates the confrontation), ADD_ITEM and SET_VARIABLE effects, a dice bluff, and three ending types.

Result: All stages pass. NuSMV confirms all three endings reachable, no deadlock or softlock, and two locked-scene invariants hold: $G(\text{node} = n_supply_closet \rightarrow \text{has_flashlight})$ and $G(\text{node} = n_confrontation \rightarrow \text{has_hidden_drive})$.

8.3 T2 - Metamodel Failure Test Cases

Two instances deliberately violate the Ecore structure. Each is rejected by the EMF validator before EVL is ever reached.

T2-01: T2_01_missing_choice_target_hll.xmi - Missing mandatory Choice.target.

The only Choice declares no target reference. Choice.target is a 1..1 EReference, so the model is structurally illegal and never reaches EVL.

Evidence (tool output):

Description: The required feature 'target' of 'Choice 1' must be set
Resource: T2_01_missing_choice_target.xmi Type: EMF Problem

T2-02: T2_02_missing_scene_id_hll.xmi - Missing mandatory Scene.id.

The start scene declares no id. Scene.id is a mandatory EString (lowerBound = 1), so EMF rejects the model at validation time.

Evidence (tool output):

Description: The required feature 'id' of 'Scene dormitory' must be set
Resource: T2_02_missing_scene_id.xmi Type: EMF Problem

8.4 T3 - EVL Failure Test Cases

Three structurally conformant models each violate domain rules. Every one passes EMF validation and is then rejected by storyflow.evl.

T3-01: T3_01_two_start_scenes_hll.xmi - Two start scenes.

Both “dorm” and “hallway” set isStart = true. The structure is legal, but EVL’s ExactlyOneStartScene constraint fails.

Evidence (tool output):

Story 'T3-01 Two Start Scenes' must have exactly one start scene, but were found: 2

T3-02: T3_02_duplicate_scene_id_hll.xmi - Duplicate scene id.

Scenes “cliff” and “lighthouse” both use id = “1”. EVL’s UniqueSceneld constraint fails for both.

Evidence (tool output):

Duplicate scene id: '1'
Duplicate scene id: '1'

T3-03: T3_03_ending_with_choice_hll.xmi - Ending scene with a choice. “

“ending_contest” is marked isEnding = true yet declares an outgoing Choice. EVL’s EndingSceneHasNoChoices constraint fails.

Evidence (tool output):

Scene 'ending_contest' is marked as an ending but has 1 outgoing choice(s). Ending scenes must be terminal.

8.5 T4 - Semantic Failure (Formal Verification) Test Cases

Two models pass EMF validation and all nine EVL constraints yet contain a behavioral fault that only the model checker can detect. These cases serve as evidence of the complementarity between static validation and formal verification: EVL reasons about one element at a time, whereas the faults below depend on the inventory and variable state accumulated across the whole reachable graph.

T4-01: T4_01_unreachable_good_ending_hll.xmi - Unreachable GOOD ending.

The GOOD ending ending_save_everyone is reachable only through a choice guarded by “inventory contains bunker_key”, but bunker_key is never produced by any action, so the guard can never be satisfied. The BAD ending ending_storm remains reachable, so the story is not deadlocked — it is simply unwinnable.

EVL cannot detect this: no constraint expresses reachability. NuSMV refutes P1 (ending reachability) and P5 (a good ending exists) while P2 and P4 hold.

Evidence (tool output):

```
-- spec EF node = n_ending_save_everyone is false
   counterexample: node = n_diner, has_bunker_key = FALSE
-- spec EF node = n_ending_storm is true
-- spec AG (!is_accepting -> can_move) is true
-- spec AG (EF is_accepting) is true
-- spec EF is_good_ending is false
-- inv !(node = n_ending_save_everyone & !has_bunker_key) is true
```

T4-02: T4_02_softlock_deadlock_hll.xmi - Softlock / deadlock.

The player walks from corridor into dark_room, whose only exit is guarded by “inventory contains exit_code” — an item the story never grants. Once inside, no transition is ever enabled again.

EVL is satisfied because dark_room does own an outgoing choice (NonEndingSceneHasChoices passes); it never checks whether that choice’s guard can fire. NuSMV refutes P2 with a trace ending in a non-ending state where can_move = FALSE.

Evidence (tool output):

```
-- spec EF node = n_ending_escape is false
-- spec AG (!is_accepting -> can_move) is false
   State 2.1 node = n_corridor can_move = TRUE
   State 2.2 node = n_dark_room can_move = FALSE
-- spec AG (EF is_accepting) is false
-- spec EF is_good_ending is false
-- inv !(node = n_ending_escape & !has_exit_code) is true
```


9. Reflection on AI Usage

9.1 Phases Where AI Was Most Useful

AI proved most useful in three distinct areas of the project:

- **Domain bootstrapping (Phase 1):** Since neither group member had played *Life is Strange*, AI was invaluable for quickly building a working understanding of the game's narrative mechanics, cross-episode variables, butterfly-effect choices, and branching endings. Rather than spending hours playing or reading wikis, we were able to describe the domain to AI and receive a structured overview that directly informed our use case scenarios.
- **Debugging Epsilon errors (Phases 4, 5 and 6):** When EVL, ETL and EGL were producing cryptic error messages, AI was a good first-line diagnostic tool. Given a clear error message and the relevant code snippet, it correctly identified the root cause in most cases. This accelerated the debug cycle compared to searching documentation alone.

9.2 Phases Where AI Was Less Useful or Produced Incorrect Output

The project confirmed the warning in the specification: LLMS are significantly weaker on niche, low-resource languages.

- **Architectural Separation:** The AI struggled with the architectural separation of our MDE pipeline. When asked to generate an HTML interface, the AI initially attempted to read data directly from the High-Level Language (HLL) metamodel (accessing Chapters and Scenes), completely bypassing the intermediate Pivot narrative graph we had established via ETL. This forced us to deeply understand our own architectural decisions to recognise that the AI's proposed solution violated the project's structural requirements.
- **ETL transformations (Phase 5):** This was the most frustrating phase for AI-assisted generation. Across multiple attempts, AI produced ETL that looked plausible but contained consistent hallucinations: invented method names, wrong rule syntax, and incorrect use of `.equivalent()`. The core issue is that ETL (Epsilon Transformation Language) has minimal representation in training data. By iteration 4–5, we had learned more about ETL from fixing the AI's mistakes than from any individual correct output. We ended up writing the rules ourselves.
- **Nusmv generation via EGL (Phase 7):** Similar issues arose with nusmv. AI had knowledge of nusmv syntax in isolation, but combining it with EGL template generation introduced compounding errors. The MODULE structure, ASSIGN blocks, and VAR declarations were often malformed. After four failed attempts we wrote the `.smv` template by hand. AI remained useful for verifying individual CTL formula logic.

- **Ecore metamodel review (Phase 2):** AI suggestions for metamodel structure were reasonable but generic. It repeatedly suggested subclassing over boolean flags, which contradicts our design decision to keep the metamodel flat. We had to be assertive in rejecting these suggestions and grounding the design in our own domain analysis.

9.3 How the Prompting Strategy Evolved

Early in the project we used zero-shot prompts expecting complete, working artifacts. This quickly proved ineffective for Epsilon-specific tasks. The first lesson was to always include the relevant metamodel structure (class names, attributes, relationships) in the prompt, rather than expecting AI to infer it. Without this, AI would invent class names. And by writing code ourselves, running it, pasting the error and code to AI, and asking for diagnosis. This played to AI's relative strength (debugging) rather than its weakness (generating novel Epsilon code from scratch).

9.4 The Role of Generative AI in Language Engineering

Our position after this project is that generative AI is both a genuine productivity tool and a real dependency risk in Language Engineering, and the balance depends heavily on which part of the SLE workflow is involved.

- For domain analysis and documentation, AI accelerates work without meaningful risk.
- For code generation in well-represented languages (HTML, javascript, Python), AI is a reliable productivity multiplier.
- For model transformation languages like ETL and EGL, and for formal verification languages like nusmv, AI is unreliable enough that over-reliance would be actively harmful to learning.

The dependency risk is subtle: AI generates confident-sounding, syntactically plausible code that only fails at runtime. This creates a false sense of progress.

10. Conclusion

This project successfully implemented a complete Model-Driven Engineering pipeline for *storyflowdsl*, an interactive branching narrative DSL grounded in the mechanical vocabulary of *Life is Strange*. The pipeline traverses seven phases: domain analysis, metamodeling (HLL and Pivot), graphical editing (Sirius), static validation (EVL), model-to-model transformation (ETL), model-to-text generation (EGL), and formal verification (nusmv). Each phase produces concrete, executable, or verifiable artifacts.

The central architectural decision, the strict separation between a human-readable HLL and a computationally oriented Pivot, proved its value at every subsequent pipeline phase. The ETL transformation is clean because the two languages have precisely defined, non-overlapping responsibilities. The EGL template is simple because the Pivot presents a flat, uniform graph. The nusmv model is tractable because the Pivot has discarded all narrative metadata that is irrelevant to state-space exploration.

The most differentiating contribution of the pipeline is the T4 test category: models that pass all nine EVL constraints but nonetheless exhibit permanently unreachable endings detectable only by model checking. This result concretely demonstrates the complementarity between static validation and formal verification, a distinction that is easy to state abstractly but that becomes vivid when an author's model passes all checks, yet the model checker returns a counterexample trace showing the "good ending" is unreachable.